



دانشگاه ملی مهارت

دانشکده فنی شهید شمسی پور

Tusk

Architecture & Code Documentation

مستندات معماری و کد اپلیکیشن مدیریت پروسه‌های ترمینالی *Tusk*، ساخته شده با *Textual* و *psutil*

(مستندات پروژه کارشناسی نرم‌افزار)

نام دانشجو: محمد سپهر فتحعلی

استاد راهنما: دکتر محمد عادل نیا

بهمن ماه 1405

Table of Contents

1. Overview
2. Tech Stack
3. Project Structure
4. Architecture & Data Flow
5. Module Reference
 - 5.1 app.py – Entry Point
 - 5.2 process/runner.py – Subprocess Execution
 - 5.3 process/manager.py – Lifecycle Orchestration
 - 5.4 process/monitor.py – Resource Monitoring
 - 5.5 ui/dashboard.py – Layout & Orchestration
 - 5.6 ui/process_panel.py – Process Table
 - 5.7 ui/log_viewer.py – Log Display
 - 5.8 ui/resource_panel.py – Resource Stats
 - 5.9 utils/helpers.py – Shared Utilities
 - 5.10 utils/logger.py – Logging Configuration
6. Key Design Patterns

1. Overview

Tusk یک اپلیکیشن رابط کاربری ترمینالی (TUI) برای مدیریت چندین پروسه shell به صورت همزمان است. این ابزار به کاربران اجازه می‌دهد دستورات shell را شروع، متوقف، راه‌اندازی مجدد و حذف کنند، همه در یک نشست ترمینال واحد، با استریم کردن خروجی به صورت لحظه‌ای، لاگ‌های رنگی و پایش منابع سیستم.

اپلیکیشن از یک لایه‌بندی سه پنلی استفاده می‌کند: لیست پروسه‌ها در سمت چپ، نمایشگر لاگ زنده در سمت راست، و پایشگر منابع سیستم در پایین صفحه. صفحه‌های مودال مبتنی بر کیبورد برای ایجاد تسک، احراز هویت sudo و ذخیره فایل لاگ استفاده می‌شوند.

- هدف اصلی Tusk ارائه یک ابزار مدیریت پروسه‌های ترمینالی است که بتواند جایگزین مناسبی برای اجرای دستی و همزمان چندین دستور shell در ترمینال‌های جداگانه باشد. در توسعه نرم‌افزار و مدیریت سرورها، بارها پیش می‌آید که نیاز به اجرای همزمان چندین پروسه داریم – مثلاً یک سرور در حال اجرا، یک اسکریپت مانیتورینگ، یک عملیات build، و یک پروسه deployment. روش سنتی باز کردن چندین تب یا ترمینال جداگانه نه تنها آشفتگی بصری ایجاد می‌کند، بلکه ردیابی خروجی هر پروسه را دشوار می‌سازد. Tusk با ارائه یک رابط کاربری یکپارچه ترمینالی تمام این پروسه‌ها را در یک صفحه واحد نمایش می‌دهد و به کاربر اجازه می‌دهد همه چیز را از یک نقطه مدیریت کند. این رویکرد بهره‌وری توسعه‌دهندگان و مدیران سرور را به طور قابل توجهی افزایش می‌دهد و از پراکندگی اطلاعات جلوگیری می‌کند.
- هدف دوم پروژه فراهم کردن تجربه‌ای روان و تعاملی برای کاربر از طریق یک رابط کاربری مبتنی بر کیبورد است که نیازی به ماوس نداشته باشد. Tusk با استفاده از فریمورک Textual ساخته شده که یک تجربه TUI غنی و مدرن فراهم می‌کند. تمام عملیات – از ایجاد تسک جدید، متوقف کردن پروسه، راه‌اندازی مجدد، حذف، تا ذخیره لاگ – تنها از طریق کلیدهای میانبر قابل انجام است. این طراحی به ویژه برای کاربران حرفه‌ای ترمینال اهمیت دارد که ترجیح می‌دهند دست‌هایشان از روی کیبورد جدا نشود. صفحه‌های مودال برای ورودی‌های پیچیده‌تر مانند دستور و رمز عبور sudo نیز با همین فلسفه طراحی شده‌اند – ساده، سریع و بدون نیاز به تعامل اضافی.
- سومین هدف پروژه ارائه پایش لحظه‌ای منابع سیستم و هر پروسه به صورت همزمان است تا کاربر بتواند تصمیمات آگاهانه‌تری درباره مدیریت پروسه‌ها بگیرد. با استفاده از کتابخانه psutil، Tusk به صورت مداوم مصرف CPU و حافظه هر پروسه در حال اجرا را رصد می‌کند و این اطلاعات را در یک پنل مجزا در پایین صفحه نمایش می‌دهد. این قابلیت به کاربر کمک می‌کند تا پروسه‌هایی که منابع بیش از حد مصرف می‌کنند را شناسایی کند و در صورت نیاز آن‌ها را متوقف یا محدود کند. همچنین نمایش CPU و حافظه کلی سیستم در کنار آمار پروسه‌ها، دید

جامعی از وضعیت سلامت سیستم ارائه می‌دهد. این پایش به صورت خودکار و بدون نیاز به دستور جداگانه انجام می‌شود.

- هدف چهارم Tusk مدیریت امن و راحت دستورات sudo است که یکی از چالش‌برانگیزترین بخش‌ها در ابزارهای ترمینالی محسوب می‌شود. بسیاری از ابزارهای مدیریت پروسه از رمز عبور sudo پشتیبانی نمی‌کنند یا کاربر را مجبور می‌کنند رمز عبور را به صورت آشکار در خط فرمان وارد کنند. Tusk با ارائه یک صفحه مودال مخصوص ورودی رمز عبور که به صورت اختیاری و فقط هنگام نیاز نمایش داده می‌شود، این مشکل را حل کرده است. رمز عبور از طریق stdin به پروسه sudo تزریق می‌شود (با استفاده از sudo -S) و هیچ‌گاه در لاگ‌ها یا تاریخچه دستورات ذخیره نمی‌شود. این رویکرد هم امنیت و هم راحتی استفاده را تضمین می‌کند.
- هدف پنجم پروژه ایجاد یک معماری نرم‌افزاری تمیز و قابل گسترش است که توسعه‌دهندگان بتوانند به راحتی قابلیت‌های جدید به آن اضافه کنند. Tusk با جداسازی clear لایه‌های مختلف – مدیریت پروسه‌ها، پایش منابع و رابط کاربری – یک معماری لایه‌ای ایجاد کرده که هر بخش مسئولیت مشخصی دارد. ماژول runner فقط اجرای ساب‌پروسه را مدیریت می‌کند، ماژول manager چرخه حیات پروسه‌ها را کنترل می‌کند، ماژول monitor پایش منابع را انجام می‌دهد، و ماژول‌های UI فقط نمایش اطلاعات را بر عهده دارند. این جداسازی با استفاده از الگوهای طراحی مانند Reactive UI، Observer/Callback و Message Bubbling پیاده‌سازی شده و اضافه کردن ویژگی‌های جدید مانند پشتیبانی از Docker، پروفایلینگ پیشرفته‌تر یا ذخیره‌سازی لاگ در دیتابیس را بدون تغییرات گسترده در کدهای موجود ممکن می‌سازد.

2. Tech Stack

Component	Technology	Purpose
Language	Python 3.10+	Primary language
TUI Framework	Textual >= 0.40.0	Terminal UI rendering, widgets, layout
System Monitoring	psutil >= 5.9.0	CPU/memory stats for system and processes
Async I/O	asyncio	Non-blocking subprocess execution and streaming
Text Rendering	Rich (bundled with Textual)	Color-coded log output, styled tables
Subprocess	asyncio.create_subprocess_shell	Shell command execution

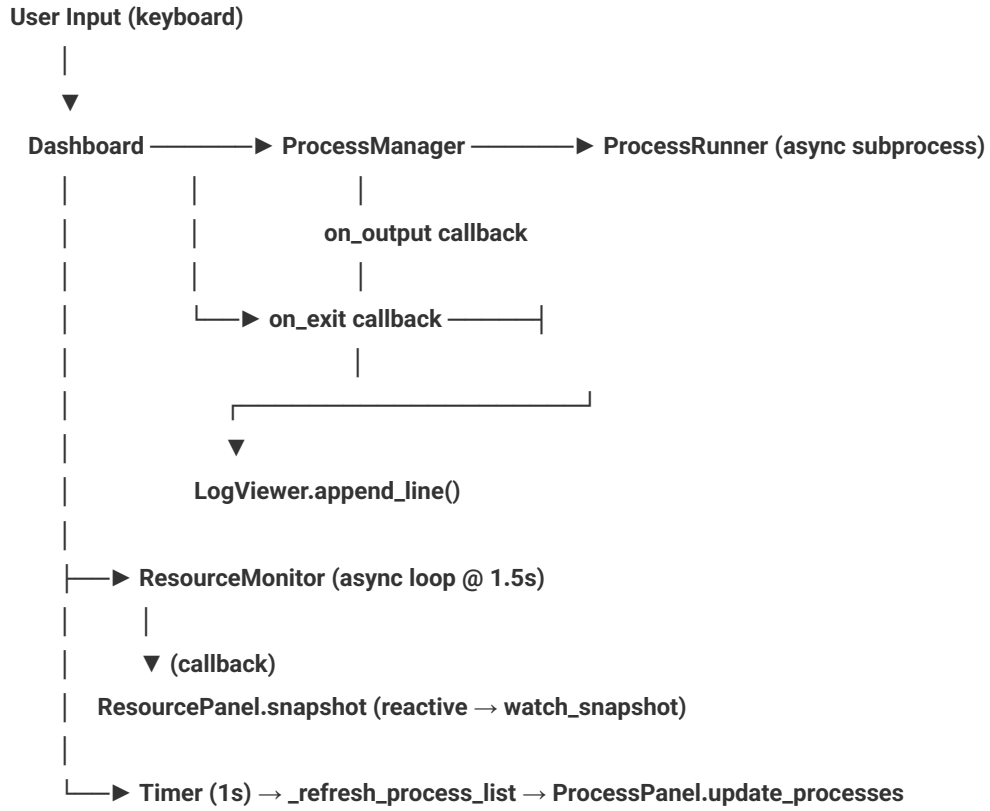
3. Project Structure

```
tusk/
├── app.py          # Application entry point
├── requirements.txt # Python dependencies
├──
├── process/        # Core process management
│   ├── runner.py   # Async subprocess execution, streaming
│   ├── manager.py  # Process lifecycle orchestration
│   └── monitor.py  # System & per-process resource monitoring
├──
├── ui/             # Textual TUI components
│   ├── dashboard.py # Main layout, keybindings, modal screens
│   ├── process_panel.py # DataTable showing process list
│   ├── log_viewer.py # RichLog with color-coded output
│   └── resource_panel.py # CPU/memory stats display
├──
└── utils/          # Shared utilities
    ├── helpers.py  # ProcessStatus enum, formatters
    └── logger.py    # Logging configuration
```

4. Architecture & Data Flow

Tusk از یک معماری لایه‌ای با جداسازی clear بین مدیریت پروسه‌ها، پایش منابع و رندر رابط کاربری استفاده می‌کند. ویجت Dashboard به عنوان هماهنگ‌کننده مرکزی عمل می‌کند و ProcessManager، ResourceMonitor و پنل‌های UI را به هم متصل می‌کند.

Data Flow Diagram



Layer Responsibilities

Layer	Module(s)	Responsibility
Entry	app.py	Creates Textual App, yields Dashboard
UI	ui/dashboard.py	Layout, keybindings, modal screens, orchestration
UI	ui/process_panel.py	DataTable of running/completed processes
UI	ui/log_viewer.py	Color-coded scrollable output log
UI	ui/resource_panel.py	CPU/memory gauges via reactive attribute
Process	process/manager.py	Lifecycle management (add/stop/restart/remove)
Process	process/runner.py	Async subprocess execution with streaming
Process	process/monitor.py	psutil-based resource polling
Utils	utils/helpers.py	ProcessStatus enum, formatting functions
Utils	utils/logger.py	Standard library logging configuration

5. Module Reference

5.1 app.py – Entry Point

ساده‌ترین ماژول در کل پروژه. کلاس `ProcessManagerApp` را تعریف می‌کند که یک زیرکلاس از `App` است و یک ویجت `Dashboard` واحد را `compose` می‌کند.

Class: `ProcessManagerApp(App)`

Member	Description
TITLE / SUB_TITLE	Window title strings displayed in the TUI header
CSS	Inline stylesheet setting the screen background to <code>\$surface</code>
<code>compose()</code>	Yields a single <code>Dashboard</code> widget as the app body

اجرا: هنگام اجرای مستقیم (`python app.py`)، اپلیکیشن را نمونه‌سازی کرده و `run()` را فراخوانی می‌کند.

5.2 process/runner.py – Subprocess Execution

پایین‌ترین سطح ماژول برای اجرای دستورات `shell`. ایجاد ساب‌پروسه ناهمگام، استریم کردن `stdout/stderr`، تزریق رمز عبور `sudo` و مدیریت وضعیت چرخه حیات را انجام می‌دهد.

Dataclass: `LogLine`

Field	Type	Description
<code>text</code>	<code>str</code>	The line of output text
<code>stream</code>	<code>str</code>	'stdout' or 'stderr'

Dataclass: ProcessInfo

Field	Type	Description
id	int	Internal process ID assigned by ProcessManager
command	str	The shell command string
status	ProcessStatus	Current status (RUNNING/COMPLETED/FAILED/TERMINATED)
pid	int None	OS-level PID once the process starts
start_time	float	time.monotonic() timestamp at creation
end_time	float None	Timestamp when process exited
exit_code	int None	Process exit code
log_lines	list[LogLine]	All captured stdout/stderr lines
runtime (property)	float	Elapsed seconds since start

Class: ProcessRunner

مدیریت یک ساب‌پروسه ناهمگام. کالک‌های اختیاری برای خروجی و رویداد خروج و رمز عبور اختیاری برای دستورات sudo می‌پذیرد.

Method	Description
run() → ProcessInfo	Core async method. Creates the subprocess via <code>asyncio.create_subprocess_shell</code> , writes sudo password to stdin if provided, reads stdout/stderr concurrently via <code>_read_stream()</code> , waits for completion, and sets final status.
_read_stream(stream, tag, info)	Async generator that reads lines from a StreamReader, decodes UTF-8, appends LogLine to <code>info.log_lines</code> , and fires the <code>on_output</code> callback.

start() → ProcessInfo	Synchronous entry point. Creates a stub ProcessInfo and kicks off run() as a background asyncio.Task. Returns the stub immediately so the UI can display the process before it finishes.
stop()	Sends SIGTERM then SIGKILL to the subprocess. Handles ProcessLookupError.
is_running (property)	Returns True if the subprocess is still active (returncode is None).

مدیریت sudo: اگر رمز عبور ارائه شود و دستور با 'sudo' شروع شود، runner دستور را به 'sudo -S' بازنویسی کرده و رمز عبور را از طریق stdin تزریق می‌کند، سپس بلافاصله stdin را می‌بندد.

5.3 process/manager.py – Lifecycle Orchestration

هماهنگ‌کننده مرکزی بین رابط کاربری و ProcessRunner. مجموعه‌ای از پروسه‌ها را بر اساس ID مدیریت می‌کند، عملیات چرخه حیات را انجام می‌دهد و اعلان‌های تغییر از طریق کالبک‌ها ارسال می‌کند.

Class: ProcessManager

Method	Description
add_process(command, on_output_line, password) → int	Creates a new ProcessRunner, starts it, stores the runner and stub ProcessInfo, and returns the assigned process ID. Wraps the output callback to also fire _notify().
stop_process(process_id)	Calls runner.stop() and updates status to TERMINATED if still RUNNING.
restart_process(process_id) → int None	Stops the old process and adds a new one with the same command. Returns the new process ID.
remove_process(process_id)	Stops the process and removes both the runner and ProcessInfo from internal dictionaries.

get_process(process_id) → ProcessInfo None	Returns ProcessInfo for a given ID, or None.
get_all_processes() → list[ProcessInfo]	Returns all tracked ProcessInfo objects.
save_log(process_id, path)	Writes command, status, exit code, runtime, and all log lines to a file.
set_update_callback(callback)	Registers a no-arg callable that fires on any state change.

مدیریت از یک شمارنده خودکار `next_id_` برای اختصاص دادن IDهای یکتا استفاده می‌کند. مکانیزم `notify_()` تضمین می‌کند که رابط کاربری هر زمان وضعیت پروسه تغییر کند، به‌روزرسانی شود.

5.4 process/monitor.py – Resource Monitoring

آمار منابع لحظه‌ای سیستم و هر پروسه را با استفاده از `psutil` فراهم می‌کند. به صورت یک حلقه ناهمگام پس‌زمینه با فاصله زمانی قابل تنظیم اجرا می‌شود.

Dataclass: ProcessStats

Field	Type	Description
cpu_percent	float	Per-process CPU usage percentage
memory_mb	float	Per-process RSS memory in megabytes
runtime	float	Seconds since process started

Dataclass: SystemStats

Field	Type	Description
cpu_percent	float	System-wide CPU usage percentage
memory_percent	float	System-wide memory usage percentage
memory_used_gb	float	Used memory in GB
memory_total_gb	float	Total memory in GB

Dataclass: MonitorSnapshot

Field	Type	Description
system	SystemStats	System-wide resource stats
processes	dict[int, ProcessStats]	Per-process stats keyed by process ID
timestamp	float	time.monotonic() snapshot time

Class: ResourceMonitor

Method	Description
track_process(process_id, pid, start_time)	Registers an OS PID for monitoring. Calls <code>cpu_percent()</code> once to initialize the psutil counter.
untrack_process(process_id)	Removes a process from monitoring.
start() / stop()	Start/stop the background async monitoring loop.
_monitor_loop()	Async loop that calls <code>_take_snapshot()</code> at the configured interval and fires the update callback.
_take_snapshot() → MonitorSnapshot	Collects system stats via <code>psutil.cpu_percent()</code> and <code>psutil.virtual_memory()</code> , then iterates tracked PIDs to get per-process CPU/memory. Handles <code>NoSuchProcess</code> and <code>AccessDenied</code> gracefully.

5.5 ui/dashboard.py – Layout & Orchestration

بزرگ‌ترین و مهم‌ترین ماژول رابط کاربری (303 خط). لایه‌بندی اصلی، تمام کی‌بایندینگ‌ها، سه صفحه مودال و هماهنگ‌سازی ارتباط بین مدیریت پروسه، پایشگر منابع و پنل‌های UI را تعریف می‌کند.

Modal Screens

سه زیرکلاس از ModalScreen برای دریافت ورودی کاربر از طریق دیالوگ استفاده می‌شوند:

Screen	Purpose	Return Value
NewTaskScreen	Command input dialog	str (command) or None (cancelled)
SudoPasswordScreen	Masked password input for sudo commands	str (password) or None (skipped)
SaveLogScreen	File path input for log saving	str (path) or None (cancelled)

Class: Dashboard(Vertical)

ویجت هماهنگ‌کننده اصلی. Header، یک کانتینر Horizontal با 55% و (ProcessPanel و (LogViewer 45%)، یک ResourcePanel در پایین صفحه و Footer را compose می‌کند.

Keyboard Bindings

Key	Action Method	Description
n	action_new_task()	Opens NewTaskScreen modal, chains to SudoPasswordScreen if needed
s	action_stop_process()	Stops the selected process via manager and monitor
r	action_restart_process()	Restarts: stops old, creates new, re-tracks in monitor
d	action_remove_process()	Removes from manager and monitor, clears log viewer
Ctrl+S	action_save_log()	Opens SaveLogScreen, saves log via manager
q	action_quit()	Stops monitor, stops all processes, exits app

Key Methods

Method	Description
on_mount()	Initializes: refreshes process list, starts 1-second periodic refresh timer, starts ResourceMonitor with callback.
_refresh_process_list()	Queries manager and monitor, passes data to ProcessPanel.update_processes(). Called on a 1-second interval timer.
_on_monitor_update(snap)	Callback from ResourceMonitor. Pushes MonitorSnapshot to ResourcePanel via its reactive snapshot attribute.
_handle_new_task(command)	Validates input, checks for sudo prefix, chains to SudoPasswordScreen or starts process directly.
_start_process(cmd, password)	Calls manager.add_process() with an on_output_line closure that forwards live lines to LogViewer for the selected process. Tracks the PID in monitor.
on_process_highlighted(event)	When user selects a row in ProcessPanel, loads that process's log into LogViewer.
action_quit()	Graceful shutdown: stops monitor, stops all processes, exits app.

5.6 ui/process_panel.py – Process Table

یک ویجت DataTable که تمام پروسه‌ها را با ستون‌های PID، Command، Status، Runtime، Exit (CPU % و Memory(MB نمایش می‌دهد.

Class: ProcessPanel(DataTable)

Method	Description
update_processes(processes, cpu_data, mem_data)	Diff-based table update: updates existing rows in-place via update_cell_at(), adds new

	rows via <code>add_row()</code> , removes stale rows. Uses <code>STATUS_COLORS</code> for colored status text.
<code>get_selected_process_id()</code> → int None	Reverse-looks up the process ID from the current cursor row.
<code>on_row_highlighted(event)</code>	Posts a <code>ProcessHighlighted</code> message when the user navigates to a different row.

پیام: `ProcessHighlighted(Message)` فیلد `process_id` را حمل می‌کند و از طریق سیستم پیام‌رسانی Textual به Dashboard بالا می‌رود (message bubbling).

5.7 ui/log_viewer.py – Log Display

یک ویجت RichLog که خروجی زنده پروسه را با رنگ‌بندی خودکار نمایش می‌دهد.

Class: `LogViewer(RichLog)`

Method	Description
<code>show_log(process_info)</code>	Clears the viewer and replays all <code>log_lines</code> from a <code>ProcessInfo</code> . Shows '(no output)' if empty.
<code>append_line(text)</code>	Appends a single line in real-time as output arrives.
<code>_write_line(text)</code>	Applies syntax-based coloring: error/fatal/panic/exception/traceback/failed/permission denied/no such file → bold red; warning/warn → yellow.
<code>save_to_file(path, process_info)</code>	Writes log content to a file with metadata header.

رنگ‌بندی با اسکن هر خط (بدون حساسیت به حروف بزرگ و کوچک) در برابر لیستی از الگوهای `regex` انجام می‌شود. الگوهای خطا اولویت دارند؛ اگر هیچ‌کدام مطابقت نداشت، الگوهای `warning` بررسی می‌شوند.

5.8 ui/resource_panel.py – Resource Stats

یک ویجت که آمار منابع سیستم و هر پروسه را نمایش می‌دهد. از سیستم `reactive Textual` برای به‌روزرسانی خودکار استفاده می‌کند.

Class: ResourcePanel(Widget)

ویژگی کلیدی: ویژگی snapshot یک `None` | `MonitorSnapshot` [reactive] است. وقتی Dashboard یک `MonitorSnapshot` جدید به آن اختصاص می‌دهد، Textual به صورت خودکار `watch_snapshot()` را برای به‌روزرسانی آمار نمایش داده شده فراخوانی می‌کند.

Element	Content
#system-stats	System CPU% and memory% on one line
#system-mem	Used/total memory in GB
#process-stats-header	Process count header
#process-stats-body	Per-process CPU% and memory in MB

5.9 utils/helpers.py – Shared Utilities

Enum: ProcessStatus

Value	String	Color
RUNNING	Running	green
COMPLETED	Completed	yellow
FAILED	Failed	red
TERMINATED	Terminated	gray

Functions

Function	Description
<code>format_duration(seconds) → str</code>	Formats seconds as 'Xh Ym Zs', 'Ym Zs', or 'Zs'.
<code>format_bytes(n) → str</code>	Converts bytes to human-readable (B/KB/MB/GB/TB/PB).
<code>elapsed_since(start) → float</code>	Returns <code>time.monotonic() - start</code> .

5.10 utils/logger.py – Logging Configuration

`setup_logger()` را برای پیکربندی لاگ‌گیری استاندارد کتابخانه Python با هندلرهای کنسول و فایل اختیاری فراهم می‌کند.

Parameter	Default	Description
name	'process_manager'	Logger name
log_file	None	Optional file path for DEBUG-level file logging
level	DEBUG	Minimum log level

این تابع idempotent است: اگر هندلرها قبلاً متصل شده باشند، یک لاگر موجود برگردانده می‌شود و از اضافه شدن هندلرهای تکراری جلوگیری می‌شود.

6. Key Design Patterns

Async I/O with asyncio

این الگو تضمین می‌کند که اپلیکیشن حتی هنگام مدیریت پروسه‌های طولانی‌مدت ترمینال، بسیار پاسخگو باقی بماند. با استفاده از `asyncio.create_subprocess_shell` همراه با `pipe`، برنامه از مسدود شدن حلقه اصلی (`main event loop`) هنگام انتظار برای خروجی ساب‌پروسه‌ها جلوگیری می‌کند. علاوه بر این، استفاده از `asyncio.gather()` به اپلیکیشن اجازه می‌دهد تا به طور همزمان به استریم‌های `stdout` و `stderr` گوش دهد، که باعث می‌شود لاگ‌های پروسه بدون ایجاد گلوگاه یا از دست رفتن داده‌ها، به صورت لحظه‌ای ثبت شوند.

Observer / Callback Pattern

این الگو برای حفظ جداسازی تمیز نگرانی‌ها (`Separation of Concerns`) بین منطق اصلی برنامه و لایه رابط کاربری حیاتی است. با استفاده از کالبک‌ها (مانند `set_update_callback`)، کلاس‌های `ResourceMonitor` و `ProcessManager` می‌توانند بدون نیاز به اطلاع از جزئیات پیاده‌سازی رابط کاربری، تغییرات وضعیت (مانند پایان یک پروسه یا به‌روزرسانی آمار منابع) را به UI اطلاع دهند. این کار باعث می‌شود منطق پس‌زمینه ماژولار، قابل تست و مستقل از رابط بصری باشد.

Reactive UI (Textual reactive)

استفاده از سیستم `reactive` در فریمورک `Textual`، به‌روزرسانی‌های UI را به طرز چشمگیری ساده می‌کند. به جای فراخوانی دستی برای رندر مجدد پرهزینه، `ResourcePanel` یک ویژگی `reactive` به نام `snapshot` تعریف می‌کند. هنگامی که `Dashboard` یک اسنپ‌شات جدید به این ویژگی اختصاص می‌دهد، فریمورک به طور خودکار متد `watch_snapshot()` را فعال کرده و تنها بخش‌های ضروری نمایشگر را به‌روز می‌کند که باعث کاهش پیچیدگی کد و سربار می‌شود.

Message Bubbling (Textual Message)

این معماری از سیستم ارتباطی رویدادمحور (`Event-driven`) فریمورک `Textual` برای جدا کردن ویجت‌های فرزند و والد استفاده می‌کند. هنگامی که کاربر یک پروسه را در `ProcessPanel` انتخاب (`highlight`) می‌کند، این پنل یک پیام `ProcessHighlighted` منتشر می‌کند. این پیام به صورت خودکار به سمت بالا ("حباب") به `Dashboard` می‌رسد که به عنوان هماهنگ‌کننده عمل می‌کند. این طراحی به ویجت‌های فرزند اجازه می‌دهد تا اقدامات سطح بالای اپلیکیشن را بدون نیاز به ارجاع مستقیم و سخت‌کد شده به والد، فعال کنند.

Diff-Based Table Updates

برای تضمین تجربه کاربری روان، ProcessPanel از استراتژی به‌روزرسانی تفاضلی (diff-based) استفاده می‌کند. به جای پاک کردن و بازسازی کل جدول در هر ثانیه، اپلیکیشن یک نداشت از شناسه‌های پروسه به ایندکس ردیف‌ها نگه می‌دارد. این سیستم هوشمندانه تشخیص می‌دهد که کدام ردیف‌ها نیاز به به‌روزرسانی دارند، کدام‌ها جدید هستند و کدام‌ها خاتمه یافته‌اند، و تغییرات را به صورت درجا (in-place) اعمال می‌کند. این کار موقعیت مکان‌نمای کاربر و تمرکز او را حفظ کرده و از "لرزش" آزادکننده جداول ترمینالی جلوگیری می‌کند.